Non-Rigid Structure from Motion

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

[1]Department of Mathematics and Informatics, FU Berlin, Berlin, DE

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

Abstract

Goal of the project: Reconstructing 3D keypoints using the NrSfM algorithm proposed in the lecture.
Main results: Matching reconstruction for Hulk data, improved reconstruction for Seq_23 and Seq_27 after normalization.
Personal key learnings: Implementation and usage of the NrSfM algorithm for 3d structure reconstruction.
Estimated working hours: 8
Project evaluation: 3
Number of Words: 846

Goal

The objective is to implement a reconstruction pipeline that estimates 3D structures from 2D keypoint observations and given camera intrinsics using a non-rigid structure-from-motion (NRSfM) approach, as introduced in the lecture. This method will be applied to three provided datasets containing 2D keypoint sequences, enabling the recovery of corresponding 3D non-rigid shapes.

Data

The data consists of 3 sets of different camera intrinsics and separate keypoint sets for each frame. First are the Seq_23 and Seq_27 datasets, where original frames come from an endoscopic recording from the inside of a colon. The Hulk dataset has frames that show the deformation of a Hulk poster. Additionally, the Hulk dataset gives us the initial depth for each frame and we can compare our 3D reconstruction to the ground truth.
First, the per-frame key points data is loaded per URL and brought into a format that then can be easily used inside the workflow. This is achieved by parsing the 2D keypoint files for each frame, transposing them to align with the expected structure in kNN, and stacking them into a single NumPy array for easier access and processing later on. Each entry in this array corresponds to a frame and contains the detected 2D key points for that specific frame. The K matrix containing the camera intrinsics was loaded and inverted to be used later on in the sightlines computation.

Methods

After importing and formatting the data, we followed the lecture's structure to implement the reconstruction method. First, we generated the neighbourhood graph (N-graph), which is used in both the geodesic distance and depth calculations. This was accomplished by running the k-nearest neighbours (KNN) algorithm on the keypoints in the first frame. We utilised the NearestNeighbors() method from the scikit-learn module for this phase.
Next, we computed the sightlines for all frames. These are direction vectors pointing from the camera's optical center to the 2D keypoints in image space, projected into 3D. To calculate them, we used the inverted intrinsic camera matrix (K-inverse) and applied it to each keypoint, as shown

in the lecture slides.

For the Hulk dataset, we were provided with initial depth values for every frame. For Seq_23 and Seq_27, we initialized all depth values to one. The geodesic distance matrix was initialized with zeros for all datasets.

We defined two functions to use later on in the iteration step. The first one computes geodesic distances based on Equation 7, and the second one updates the depth values using Equation 11. The geodesic function computes distances between each point and its neighbors by averaging across all frames. The depth update function uses those geodesics and the sightlines to adjust the current depth estimates. Additionally, for the depth computation, we included a learning rate, as otherwise it produced numerical issues.

For the iteration loop, we set a maximum iteration number of 42 and a convergence value of 0.0001. These signify whether the iteration loop stops when the difference between the steps is smaller than the convergence value or it reaches the maximum iterations allowed. One iterative step goes as follows: compute geodesics based on current depths, compute depths based on current geodesics, check for convergence, and update depth if not converged.

This procedure worked well for the Hulk data, but for the Seq_27 and Seq_23, we introduced a geodesics normalization step between computing geodesics and computing depths, as the reconstructed structure looked flat compared to the expected output of a curved colon. Therefore, a normalization step between the geodesics and depth update was implemented, which normalizes the geodesics with the Frobenius norm, which standardizes the scale of the geodesic distances matrix while preserving their relative structure.

$$\mathrm{d}_{norm} = \frac{\mathrm{d}}{\sqrt{\sum_{i,j} \mathrm{d}_{ij}^2}}$$

After the depth calculation, we reconstructed the 3D data by multiplying the sightlines with the depths and plotted the outcomes.

Results & Discussion

As a reference that everything was implemented correctly, we compare the reconstructed Hulk 3D data with the provided ground truth. As seen in ***Figure A3***(middle and bottom rows) for the Hulk data, our implementation provides an accurate reconstruction that visually aligns with the ground truth across the tested frames. The structure, orientation, and relative distances between keypoints match closely, validating the correctness of our workflow.

For the Seq_23 and Seq_27 datasets, the 3D reconstructions show the expected curvature of a colon (***Figure A1*** and ***Figure A2*** middle row), as seen in the endoscopic imaging in the lecture slides. While no ground truth is available for these sequences, the shape across frames suggests that the structure is plausibly recovered.

Additionally, for the colon endoscopy Seq_23 and Seq_27 data, we saw that adding a normalization step before depth computation improved the reconstruction and resulted in a better curved structure . When we tried reconstructing the Seq_23 and Seq_27 datasets without this normalization step, the 3D shapes looked squished, warped, or too flat (***Figure A1*** and ***Figure A2*** bottom row). This likely happened because some 2D points in the raw data were too far apart or not evenly distributed (***Figure A1*** and ***Figure A2*** top row), which caused problems during the iterative updates.

Contributions

1. Knn, geodesics, depth, framewise 3d reconstruction and application on Seq_27 data were implemented by Ksenia Krasikova.
2. Hulk and Seq_23 applications were implemented by Payal Priyadarshini
3. Report writing and minor code assistance

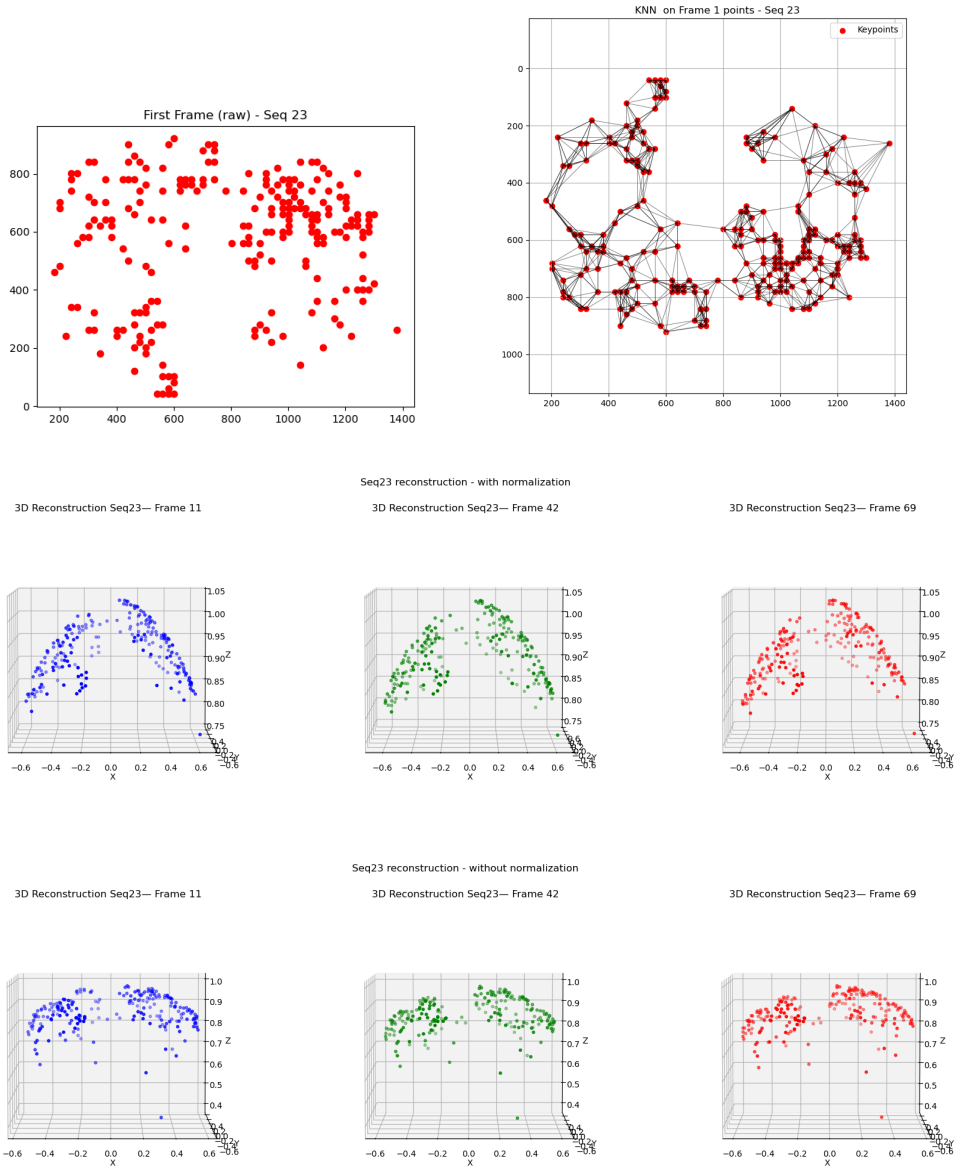
References

Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, Berg S, Smith NJ, Kern R, Picus M, Hoyer S, van Kerkwijk MH, Brett M, Haldane A, del Río JF, Wiebe M, Peterson P, Gérard-Marchant P, et al. Array programming with NumPy. Nature. 2020 Sep; 585(7825):357–362. http://dx.doi.org/10.1038/s41586-020-2649-2, doi: 10.1038/s41586-020-2649-2.

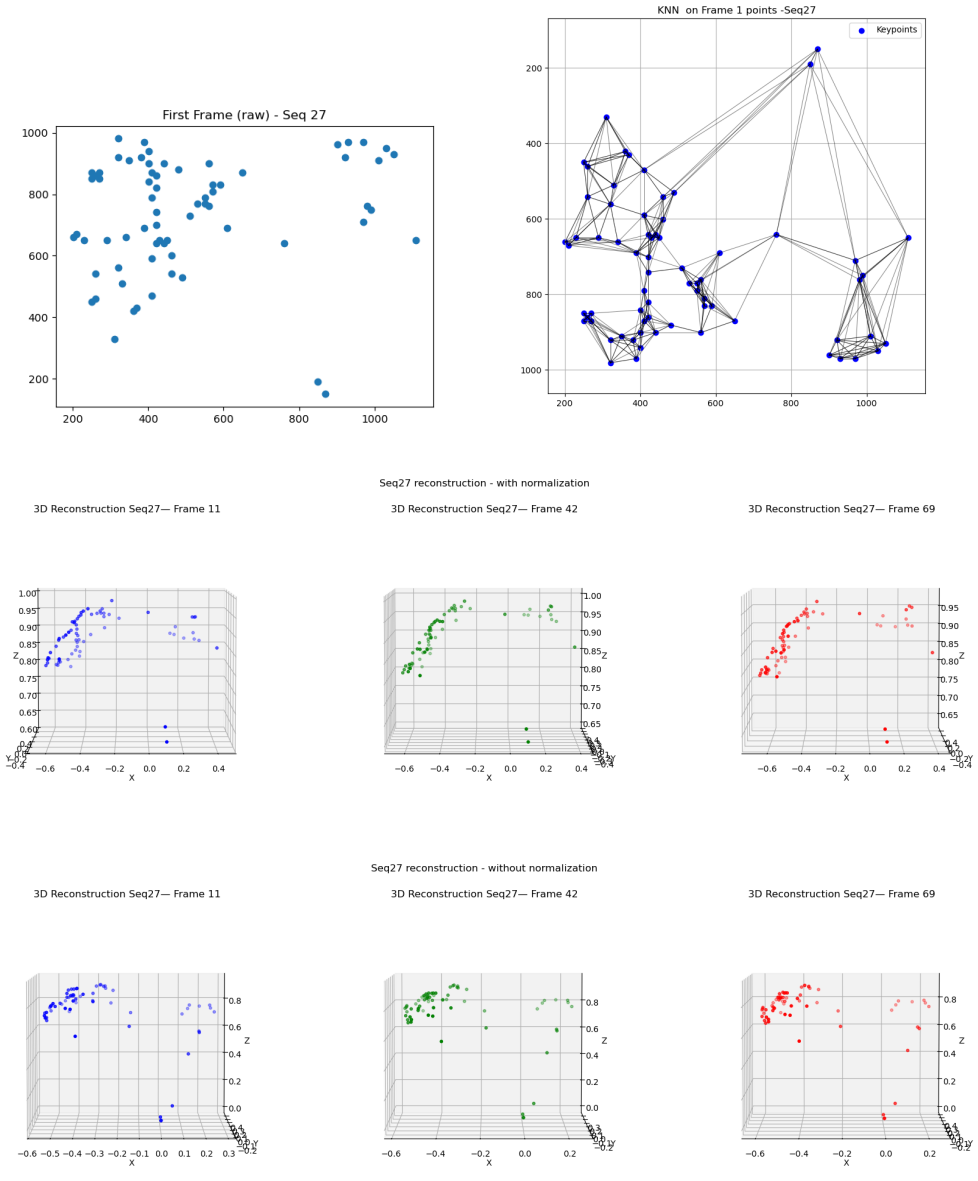
Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Müller A, Nothman J, Louppe G, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Édouard Duchesnay, Scikit-learn: Machine Learning in Python; 2018. https://arxiv.org/abs/1201.0490.

Sengupta A, Deformable 3D Reconstruction; 2025. Lecture slides, SOSE 2025 Geometric Computer Vision course. https://git.zib.de/asengupta/sose2025_geometric_cv.

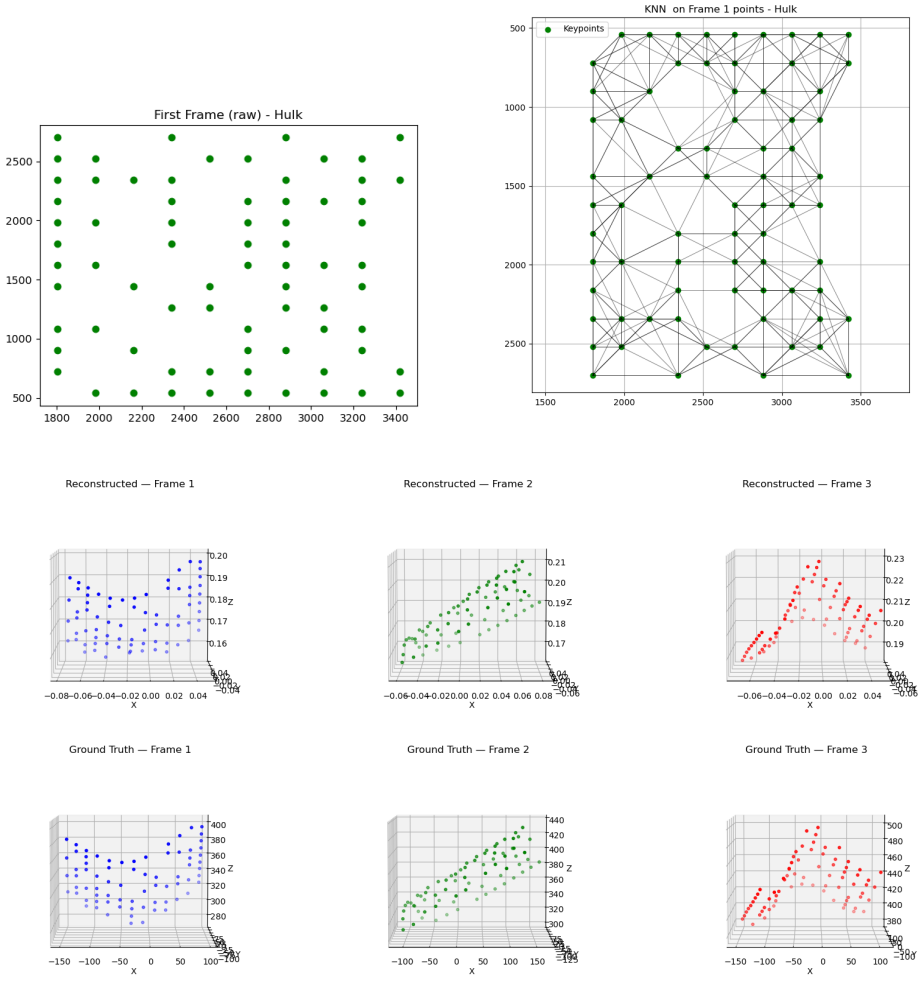
Supplementary Figures



Appendix 0—figure A1. 3D Reconstruction of Seq23. Top row: raw 2D points and kNN graph of the first frame. Middle row: 3D reconstruction with normalization. Bottom row: 3D reconstruction without normalization.



Appendix 0—figure A2. 3D Reconstruction of Seq27. Top row: raw 2D points and kNN graph of the first frame. Middle row: 3D reconstruction with normalization. Bottom row: 3D reconstruction without normalization.



Appendix 0—figure A3. 3D Reconstruction of Hulk data. Top row: raw 2D points and kNN graph of the first frame. Middle row: 3D reconstruction. Bottom row: Ground truth 3d reconstruction.